

Assignment

On

Computer Networks

Submitted To

Md. Sakhawat Hossain

Lecturer,

Department of Computer Science and Engineering

Khulna University of Engineering & Technology,

Khulna

Submitted By

Syeda Hafsa Tazrian

Roll: 2007072

Section: B

Batch: 2020

CSE, KUET

Question A: Implement Dijkstra's Algorithm Using Heap. Give your verdict on it.

Solution:

In a weighted graph, Dijkstra's algorithm is a popular algorithm to find the shortest path from a source node to all the other nodes. It runs in $O(V^2)$ time. But using a heap reduces the time complexity to $O(V + E)$ where V denotes the vertices and E denotes the edges. So using heap makes the algorithm run faster as it ensures that for each step the node which has the smallest distance is chosen.

Dijkstra's algorithm implemented with heap is ideal for very large or sparse graphs where $E \ll V^2$. For example network routing protocols or map navigation systems (GPS) can benefit from this. A minor issue is that a vertex may be inserted multiple times with same distance values into the heap, which takes more memory. So sometimes a memory trade-off happens in heap-based dijkstra's algorithm.

Algorithm:

Step 1: Initialization

- i. Create a heap (Min-heap) which stores pairs (distance, vertex).
- ii. Set distance to source vertex as 0, and to all the other vertices as infinity.
- iii. Insert the source vertex (distance 0) into the heap.
- iv. Mark all vertices as unvisited.

Step 2: Main Loop

While heap is not empty:

- i. Extract the vertex u with minimum distance (at the top of heap).
- ii. Mark vertex u as visited vertex.
- iii. For every neighboring vertex v of u which has not been visited:
 - a) Calculate new distance: $\text{new_dist} = \text{distance}[u] + \text{weight}(u,v)$
 - b) If new distance $< \text{distance}[v]$:
 - Update $\text{distance}[v] = \text{new_dist}$.
 - Insert v with new distance value into heap.
 - Store u as the parent of v .

Step 3: Termination

- i. When heap becomes empty, the algorithm is terminated. The dist vector contains the shortest path from the source to every node.

Question B: While implementing distance vector routing using Bellmanford, count-to-infinity problem occurs. How to solve it?

Solution:

In distance vector routing, routers share their distance vectors periodically with neighboring routers. It uses Bellman-Ford algorithm where each router maintains a table of distances to all destinations. But if a link fails, the router continues to send outdated information to and from, which increases their distance indefinitely, which is known as count-to-infinity problem.

Suppose there are three routers A, B, C which are connected in a way such that A—B—C. At the beginning, B knows it can reach C with cost 1 and so A learns from B that it can reach C in cost 2. But if B—C line fails, B immediately can not know that C is unavailable and sees that A can reach C in cost 2, so B updates route that it can reach C in cost 3. A sees this and updates route such that it can reach C in cost 4. This continues indefinitely and results to count-to-infinity problem.

Poison Reverse solves this count-to-infinity problem. It tells a router not to advertise a route back to the neighbor it itself learned it from.

Algorithm:

Step 1: Initialization:

- i. Each router maintains a distance vector for all destinations.
- ii. Set distance to itself as 0, to direct neighbors as measured cost and to all others as infinity.

Step 2: Periodic Updates:

- i. Create an advertisement vector.
- ii. For each destination D:
 - If $\text{next_hop}[D] = N$, then advertise distance to D as infinity (poison the route).
 - Else advertise actual distance to D.
- iii. Send the customized vector to N.

Step 3: Receiving Updates

- i. Receive the distance vector from neighbor N.
- ii. For each destination D in the received vector, update $\text{distance}[D] = \text{new_distance}$ and $\text{next_hop}[D] = N$.
- iii. If $\text{next_hop}[D] == N$ and distance has changed, then update the $\text{distance}[D]$ accordingly.

Step 4: Triggered Updates: When a route changes (especially failures):

- Immediately send the poison route to the router which was using it.
- Send the updated vectors to all the neighbors.
- No waiting for the timer.

Though Poison reverse works perfectly for two-node loops but it has problems with three or more router forming a routing loop.